RAPPORT DE PROJET POO

Métadonnées de fichiers MP3 et playlists

Projet POO & Java L2-I 2025-2026

Groupe projet C05

Réalisé par:

Meriem SIMYA & Ines LAIFAOUI

Sommaire:

1. Introduction: Contexte du projet et démarche adoptée
2. Architerture et choix de conception:
    - **2.1** Package core
    - **2.2** Package playlist
    - **2.3** Package cli
    - **2.4** Package gui
3. Mode console (CLI): Implémentation et difficultés rencontrées:
    - **3.1** Organisation du mode CLI
    - **3.2** Problème majeur: confusion entre analyse et export
    - **3.3** Gestion des chemins et des fichiers MP3
    - **3.4** Problème des doublons lors de l'export
4. **Mode Graphique (GUI): Conception et ajustements:**
    - **4.1** Objectifs du mode graphique
    - **4.2** Affichage des métadonnées: correction nécessaires
    - **4.3** Gestion de l'export des playlists en GUI
    - **4.4** Ajout à une playlist existante
5. Tests et validation
6. Bibliothèque utilisées
    - **6.1** Bibliothèques Java internes
    - **6.2** Bibliothèque externe: JAudiotagger
7. Répatition du travail au sein du binôme
8. Bilan critique
9. Conclusion

1. Contexte du projet et démarche adoptée

Dans le cadre de ce projet, nous avons développé une application Java permettant de manipuler des fichiers MP3, d'en extraire les métadonnées et de générer des playlists selon différents formats. Le développement s'est fait de manière **itérative**, en alternant implémentation, tests concrets et corrections successives.

Notre démarche a été guidée par deux objectifs principaux :

- construire une application **fonctionnelle et conforme** aux attentes,
- mettre en pratique une **conception orientée objet propre**, cohérente avec les notions vues en cours (responsabilités, modularité, polymorphisme).

Le travail a été réalisé en binôme (Inès et Meriem), avec une répartition des tâches flexible : la conception UML, le mode console et le mode graphique ont été développés en parallèle, puis ajustés pour rester cohérents entre eux.

2. Architecture et choix de conception

Dès le départ, nous avons choisi de structurer le projet en plusieurs packages afin de séparer clairement les responsabilités :

2.1 Package core

Le package core regroupe les classes liées à la **gestion des fichiers MP3**, à l'extraction des métadonnées et à l'exploration du système de fichiers.

FichierMP3

Cette classe représente un fichier MP3 individuel. Elle encapsule :

- le nom du fichier,
- son chemin absolu,
- ses métadonnées.

Elle fournit des méthodes permettant :

- de charger les métadonnées à partir du fichier physique,
- d'accéder aux informations musicales (titre, artiste, album, année, genre),
- de récupérer l'image de couverture si elle existe.

Cette classe constitue l'élément central manipulé aussi bien en mode CLI qu'en mode GUI.

MetaDonnees

La classe MetaDonnees représente les informations descriptives d'un fichier MP3. Elle contient notamment :

- le titre,
- l'artiste,
- l'album,
- l'année,
- le genre,

- l’image de couverture.

Elle permet une séparation claire entre les **données musicales** et la **logique de traitement** du fichier MP3, ce qui simplifie l’affichage et l’export.

ExplorateurDeRepertoire

Cette classe est responsable de l’exploration récursive d’un dossier. À partir d’un chemin donné, elle :

- parcourt l’arborescence de dossiers,
- détecte les fichiers MP3,
- construit une structure logique représentant le contenu du répertoire.

Elle est utilisée aussi bien :

- en mode CLI (option -d),
- qu’en mode GUI lors de la sélection d’un dossier.

Dossier

La classe Dossier représente un répertoire analysé. Elle stocke :

- la liste des fichiers MP3 qu’il contient,
- ainsi que ceux présents dans ses sous-dossiers.

Elle fournit une méthode de récupération récursive permettant d’obtenir l’ensemble des fichiers MP3 contenus dans l’arborescence.

2.2 Package playlist

Ce package regroupe les classes liées à la **création et à l’export de playlists**.

Playlist

La classe Playlist représente une playlist logique composée de plusieurs fichiers MP3. Elle permet :

- d’ajouter des morceaux,
- de stocker leur ordre,
- de transmettre ces informations aux classes d’export.

FormatPlaylist (interface)

Cette interface définit un contrat commun pour tous les formats de playlist. Elle impose la méthode :

- exporter(Playlist playlist, String chemin)

Ce choix de conception permet l’utilisation du **polymorphisme**, facilitant l’ajout de nouveaux formats sans modifier le reste de l’application.

FormatXSPF, FormatJSPF, FormatM3U8

Ces classes implémentent l’interface FormatPlaylist. Chacune correspond à un format spécifique de playlist et contient la logique nécessaire pour :

- écrire les fichiers de sortie,
- respecter la syntaxe propre au format concerné.

2.3 Package cli

Le package cli regroupe les classes dédiées à l'exécution en **ligne de commande**.

ApplicationCLI

Cette classe constitue le point d'entrée du mode console. Elle orchestre :

- l'analyse des arguments,
- l'exploration des fichiers ou dossiers,
- l'affichage des métadonnées,
- et l'export éventuel de playlists.

Un travail important a été réalisé pour distinguer clairement :

- les commandes d'analyse seule,
- des commandes combinant analyse et export.

GestionArgumentCLI

Cette classe est responsable du décodage et de la validation des arguments passés en ligne de commande. Elle permet :

- d'identifier le mode demandé (fichier, dossier, export),
- d'éviter des incohérences (format absent, chemin manquant),
- de centraliser la logique de validation.

2.4 Package gui

Le package gui contient les classes liées à l'interface graphique Swing.

ApplicationGUI

Cette classe gère l'ensemble de l'interface graphique :

- boutons de sélection de fichiers et dossiers,
- affichage de la liste des morceaux,
- affichage détaillé des métadonnées,
- visualisation de la couverture,
- export des playlists avec choix du nom et du chemin.

Elle repose sur les classes du package core et playlist, sans dupliquer la logique métier.

SelectionFichierGUI

Cette classe encapsule les opérations de sélection graphique (fichiers et dossiers) à l'aide de JFileChooser, afin de simplifier le code de l'interface principale.

Ce découpage nous a permis :

- d'éviter le mélange entre interface utilisateur et logique métier ;
- de faciliter les tests (notamment en CLI) ;
- de faire évoluer le GUI sans modifier le cœur de l'application.

Le diagramme de classes a été ajusté plusieurs fois au cours du projet afin de refléter fidèlement l'implémentation réelle, notamment après la résolution de certains problèmes liés à l'export et à la gestion des métadonnées.

3. Mode Console (CLI) : implémentation et difficultés rencontrées

3.1 Organisation du mode CLI

Le mode console repose sur une classe principale (ApplicationCLI) qui orchestre l'exécution à partir des arguments fournis par l'utilisateur. Une classe dédiée (GestionArgumentCLI) se charge de l'analyse et de la validation de ces arguments.

Cette séparation s'est révélée essentielle lorsque nous avons commencé à gérer plusieurs scénarios différents (analyse simple, analyse + export).

3.2 Problème majeur : confusion entre analyse et export

L'un des principaux problèmes rencontrés concernait l'erreur suivante :

Cannot invoke "String.hashCode()" because "opt.format" is null

Cette erreur apparaissait lorsque nous exécutons une commande destinée **uniquement à l'analyse** (par exemple -d .), sans intention d'exporter une playlist.

Cause identifiée

Le code exécutait systématiquement la partie "export" après l'analyse, même lorsque l'utilisateur n'avait pas fourni de format (--xspf, --jspf, --m3u8). Ainsi, la variable opt.format était null, mais utilisée dans un switch.

Solution mise en place

Nous avons séparé clairement les deux comportements :

- **analyse seule** : affichage des fichiers et/ou des métadonnées, puis arrêt ;
- **analyse + export** : uniquement si un format et un fichier de sortie sont fournis.

Cette correction a permis de rendre le CLI conforme aux scénarios attendus et d'éviter des erreurs inutiles.

3.3 Gestion des chemins et des fichiers MP3

Un autre point délicat a été la gestion des chemins de fichiers, notamment sous Windows :

- chemins relatifs mal interprétés ;
- chemins contenant des espaces non protégés par des guillemets ;
- fichiers stockés dans OneDrive avec des problèmes de droits d'accès.

Ces situations provoquaient des erreurs du type :

- Unable to find: fichier.mp3
- métadonnées affichées comme null alors que le fichier en contenait.

Nous avons compris que ces erreurs ne venaient pas du code d'extraction des métadonnées, mais :

- soit d'un chemin invalide,
- soit d'un fichier réellement inaccessible.

Cette phase nous a appris l'importance de **tester sur de vrais cas utilisateurs** et pas uniquement dans Eclipse.

3.4 Problème des doublons lors de l'export

Lors de l'export d'un dossier, nous avons observé que chaque fichier apparaissait deux fois dans la playlist générée.

Analyse

Le problème venait d'un ajout multiple des fichiers :

- une première fois lors de l'analyse récursive du dossier,
- une seconde fois lors de la construction de la playlist.

Correction

Nous avons veillé à ce que chaque fichier ne soit ajouté qu'à un seul endroit du flux d'exécution, ce qui a supprimé les doublons.

4. Mode Graphique (GUI) : conception et ajustements

4.1 Objectifs du mode graphique

Le mode graphique a été conçu pour offrir une utilisation plus intuitive :

- sélection visuelle des fichiers et dossiers ;
- affichage structuré des métadonnées ;
- génération de playlists sans passer par la ligne de commande.

L'interface repose sur Swing et s'articule autour :

- d'une liste de fichiers MP3 ;
- d'une zone d'affichage des métadonnées ;
- d'une zone d'affichage de la couverture ;
- de boutons d'export.

4.2 Affichage des métadonnées : correction nécessaire

Un problème important concernait l'affichage des informations dans la liste de gauche. Initialement, la liste affichait des informations trop détaillées (chemin, métadonnées), ce qui n'était pas souhaité puisque les métadonnées étaient déjà affichées à droite.

Cause

Swing affiche par défaut le résultat de la méthode toString() des objets.

Décision

Nous avons limité le rôle de la liste de gauche à une **simple sélection de fichiers**, et réservé l'affichage détaillé des métadonnées à la zone dédiée à droite. Cela a rendu l'interface plus claire et conforme à l'usage attendu.

4.3 Gestion de l'export des playlists en GUI

Problème rencontré

L'export écrasait systématiquement les playlists précédentes, car le nom et le chemin étaient fixés dans le code.

Solution

Nous avons intégré un JFileChooser en mode sauvegarde afin de permettre à l'utilisateur :

- de choisir le nom du fichier,
- de choisir le dossier de destination.

Cela a considérablement amélioré l'ergonomie de l'application.

4.4 Ajout à une playlist existante

Une difficulté supplémentaire a été la gestion de l'ajout de morceaux à une playlist déjà créée.

Techniquement, ajouter à une playlist existante implique de :

- relire le fichier de playlist,
- reconstituer la structure,
- ajouter les nouveaux morceaux,
- réécrire le fichier.

Dans le cadre du projet et au niveau L2, nous avons opté pour une solution pragmatique :

- permettre de créer une nouvelle version de la playlist mise à jour,
- sans altérer l'ancienne.

5. Tests et validation

Nous avons testé l'application :

- avec différents fichiers MP3 (avec et sans métadonnées) ;
- avec des dossiers contenant plusieurs niveaux de sous-dossiers ;
- en CLI et en GUI ;
- avec des chemins absolus et relatifs.

Ces tests nous ont permis de mettre en évidence des erreurs non visibles au premier abord et d'améliorer la robustesse globale de l'application.

6. Bibliothèques utilisées

6.1 Bibliothèques Java internes

Le projet utilise plusieurs bibliothèques standards de Java :

- java.io : gestion des fichiers et flux,
- java.util : listes, collections, itérations,
- javax.swing : interface graphique,
- java.awt : gestion graphique et images.

Ces bibliothèques assurent la portabilité et la compatibilité de l'application.

6.2 Bibliothèque externe : JAudiotagger

Pour l'extraction des métadonnées MP3, nous avons utilisé la bibliothèque **JAudiotagger**.

Elle permet :

- la lecture des tags ID3 (ID3v1, ID3v2),
- l'accès aux champs standards (titre, artiste, album, etc.),
- l'extraction de l'image de couverture.

L’utilisation de cette bibliothèque nous a permis de nous concentrer sur la logique applicative sans réimplémenter des mécanismes complexes de lecture binaire des fichiers MP3.

7. Répartition du travail au sein du binôme

Notre projet a été réalisé en collaboration entre **Ines** et **Meriem**, avec une répartition équilibrée des tâches :

- **Ines**
  - conception et ajustement du diagramme UML,
  - développement principal du mode CLI,
  - gestion des erreurs et corrections liées aux arguments,
  - participation active au mode GUI et aux tests,
  - rédaction du rapport et documentation.
- **Meriem**
  - recherche sur les formats MP3 et playlists,
  - implémentation des formats XSPF / JSPF / M3U8,
  - développement initial du mode GUI,
  - tests fonctionnels et validation des exports,
  - participation à la documentation et aux démonstrations.

Le diagramme de Gantt illustre cette répartition et montre une progression cohérente du projet sur plusieurs semaines

8. Bilan critique

Points forts

- architecture claire et modulaire ;
- séparation nette entre logique métier, CLI et GUI ;
- gestion de plusieurs formats de playlists via polymorphisme ;
- interface graphique fonctionnelle et intuitive.

Points à améliorer

- import complet de playlists existantes ;
- meilleure gestion des erreurs utilisateurs en CLI ;
- optimisation de la gestion des chemins et des permissions ;
- possibilité future de lecture audio.
-

9. Conclusion

Ce projet nous a permis de consolider nos compétences en programmation orientée objet, en conception UML et en développement Java. Les nombreuses erreurs rencontrées ont été formatrices et nous ont appris à analyser un problème, identifier sa cause réelle et proposer une solution adaptée.

Le résultat final est une application fonctionnelle, cohérente et évolutive, qui répond aux objectifs fixés tout en mettant en évidence les enjeux pratiques du développement logiciel.